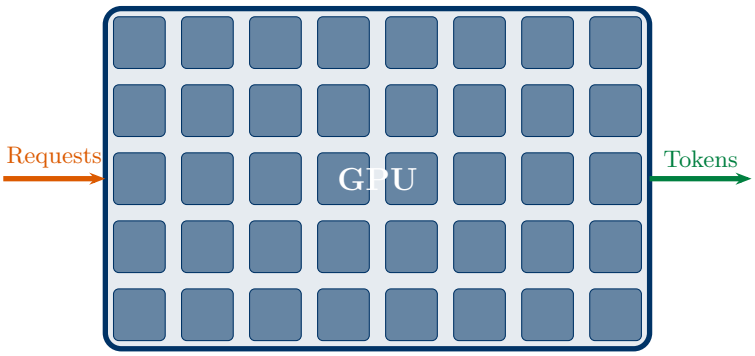
AI Inference Engineering

Building Fast, Scalable, and
Production-Ready AI Systems

Author: Shivam Kumar

AI Inference Engineering

Building Fast, Scalable, and Production-Ready AI Systems



From First Principles to Production Systems

Author: Shivam Kumar
Website: https://shivam2003.com

Covers: GPU Architecture · CUDA · Transformers · KV Cache · Quantization
vLLM · TensorRT-LLM · SGLang · Speculative Decoding · Distributed Inference
Kubernetes · Observability · NVIDIA H100/Blackwell · 2025–2026

AI Inference Engineering: Building Fast, Scalable, and Production-Ready AI Systems

Edition 1.0 — May 2026

Author: Shivam Kumar
Website: https://shivam2003.com

This book covers GPU inference engineering from first principles through production deployment. It incorporates the latest developments in NVIDIA Blackwell architecture (B200/GB200), vLLM PagedAttention, TensorRT-LLM v1.2, SGLang, speculative decoding, FP8/FP4 quantization, and NVIDIA Dynamo disaggregated serving.

For educational purposes.

Typeset with LaTeX / TeX Live 2023.

Contents

Preface

"The best way to learn is to teach."

This book exists because AI is no longer just a research topic — it is an engineering discipline. Every time you ask a question to ChatGPT, Claude, or Gemini, thousands of GPU calculations happen in milliseconds. Behind that seamless experience is a carefully engineered system: one that balances speed, cost, memory, and reliability at a massive scale.

AI Inference Engineering teaches you how to build and operate those systems.

Who this book is for

- Software engineers wanting to understand AI infrastructure
- ML engineers who want to go beyond model training
- Platform engineers deploying AI at scale
- Students learning GPU systems and distributed computing
- Anyone curious about what happens after a model is trained

What you will learn

You will journey from the very first question — *what is a GPU?* — all the way to operating a multi-node, multi-GPU inference cluster serving millions of requests per day. Every concept is built from scratch using simple language and real-world analogies before advancing to production-grade engineering detail.

How to read this book

Read it linearly the first time. Each chapter builds on the previous one. Revisit individual chapters as reference when working on production systems. Run the code examples — intuition comes from doing.

A note on versions

The field moves fast. This edition reflects the state of the art as of mid-2026: NVIDIA H100 and Blackwell (B200/GB200) GPUs, vLLM v0.7+, TensorRT-LLM v1.2, SGLang v0.4+, and Kubernetes-based deployment patterns. Core concepts — batching, memory management, quantization, parallelism — are timeless.

PART I

Foundations

CHAPTER 1

Why Inference Engineering Matters

Learning Objectives

- Understand the difference between AI training and AI inference
- Learn why inference efficiency is the key challenge for production AI
- Grasp the scale of modern AI inference systems
- Appreciate the engineering disciplines that make AI products work

1.1 The Two Lives of an AI Model

Every AI model lives two lives.

In its **first life**, the model learns. Enormous clusters of GPUs process billions of text samples, adjusting billions of numerical parameters over weeks or months. This is called **training**.

In its **second life**, the model works. It answers questions, writes code, summarizes documents, and assists doctors, lawyers, and students around the world. This is called **inference**.

Training is a one-time event. Inference happens billions of times a day.

Analogy: The Chef and the Restaurant

Imagine a chef who spends years learning to cook — practising recipes, mastering techniques, developing a personal style. That long learning period is *training*.
Now the restaurant opens. Every day the chef must cook hundreds of meals, each one served in minutes. That daily cooking is *inference*.
The chef trains once. The restaurant runs forever.

1.2 The Inference Problem

When an AI model answers a question, it performs billions of mathematical operations for every sentence it generates. A single forward pass through a 70-billion-parameter model involves roughly 140 billion floating-point multiplications.

Now multiply that by the users:

- ChatGPT processes over 100 million queries per day.
- A single query might generate 200–1000 tokens.
- Each token requires a full model forward pass.

This means AI companies run **trillions** of mathematical operations every second just to keep their services alive. If those operations are not performed efficiently, one of two things happens: the service becomes unbearably slow, or the cost becomes unbearably high.

Key Insight

NVIDIA's 2025 Annual Report explicitly noted that "inference workloads surpassed training" in revenue terms. The industry has shifted: getting models to run efficiently is now more important commercially than making

them more accurate.

1.3 What Inference Engineers Do

An inference engineer's job is to make AI fast, cheap, and reliable at scale. This involves:

- **Hardware selection:** Which GPUs to buy or rent, and how many
- **Runtime optimization:** Making each GPU operation as fast as possible
- **Memory management:** Fitting large models into limited GPU memory
- **Batching:** Serving many users simultaneously from one GPU
- **Quantization:** Compressing model weights to use less memory and run faster
- **Deployment:** Packaging and scaling inference services on Kubernetes
- **Observability:** Monitoring latency, throughput, and cost in production

1.4 The Three Metrics That Matter

Every inference system is measured by three core metrics:

Latency How long does the user wait? Specifically, *Time to First Token (TTFT)* — the delay before the first word appears — and *Time per Output Token (TPOT)* — how quickly subsequent words arrive.

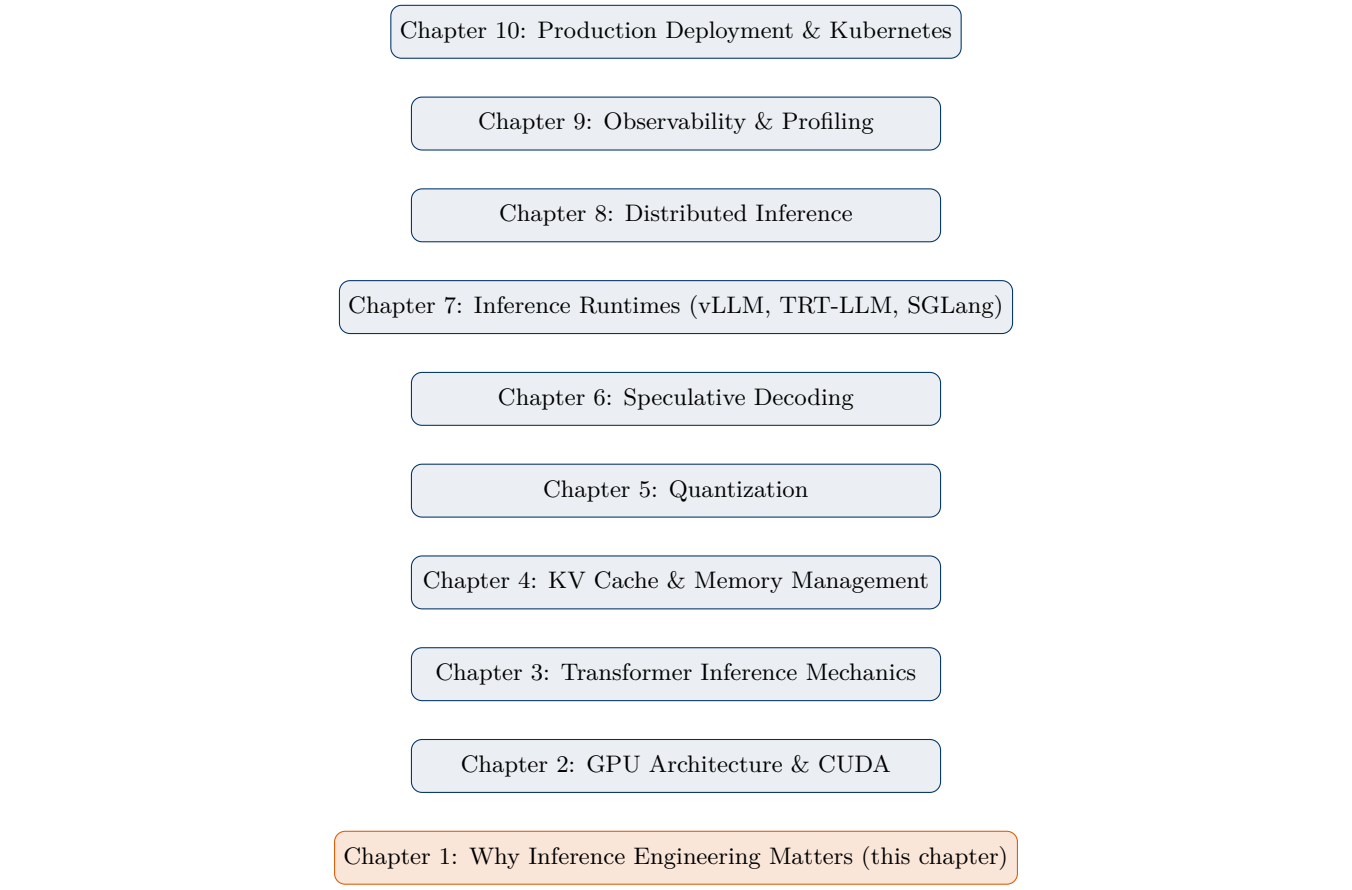
Throughput How many requests can the system process per second? Higher throughput means lower cost per query.

Memory How much GPU memory does the system use? Memory limits how large a model you can serve and how many requests you can handle simultaneously.

These three metrics form a **triangle of tradeoffs**. Improving one often hurts another. A large batch size improves throughput but increases latency. Quantization reduces memory but may reduce accuracy. Understanding these tradeoffs is the core skill of an inference engineer.

1.5 A Preview of the Journey

This book walks through the entire inference engineering stack, layer by layer:



Each layer depends on the one below it. Build a firm foundation and the rest becomes intuitive.

Chapter Summary

AI inference is the act of running a trained model to answer user queries. It is performed billions of times daily and requires careful engineering to be fast and affordable. Three key metrics — latency, throughput, and memory — govern every decision an inference engineer makes. This book builds your knowledge from the GPU chip upward to a full production deployment.

Exercise 1.1: Reflection

Before reading further, write down three AI products you use daily. For each one, estimate: How many users might use it simultaneously? What latency would feel "good enough" to you? Keep these numbers in mind as you read — they will help you appreciate why every optimization in this book matters.

CHAPTER 2

GPU Architecture and CUDA Fundamentals

Learning Objectives

- Understand the fundamental difference between CPUs and GPUs
- Learn GPU memory hierarchy: registers, shared memory, HBM
- Understand CUDA programming model: threads, blocks, grids
- Grasp how Tensor Cores accelerate matrix operations
- Understand NVIDIA's GPU generations: Volta through Blackwell

2.1 Why GPUs? The Parallel Computation Story

A CPU (Central Processing Unit) is designed for sequential, versatile work. It has a small number of very powerful cores — typically 8 to 64 — each capable of complex decision-making, branch prediction, and running operating systems.

A GPU (Graphics Processing Unit) is built for a completely different purpose: doing the *same simple operation thousands of times simultaneously*.

Analogy: One Expert vs. One Thousand Students

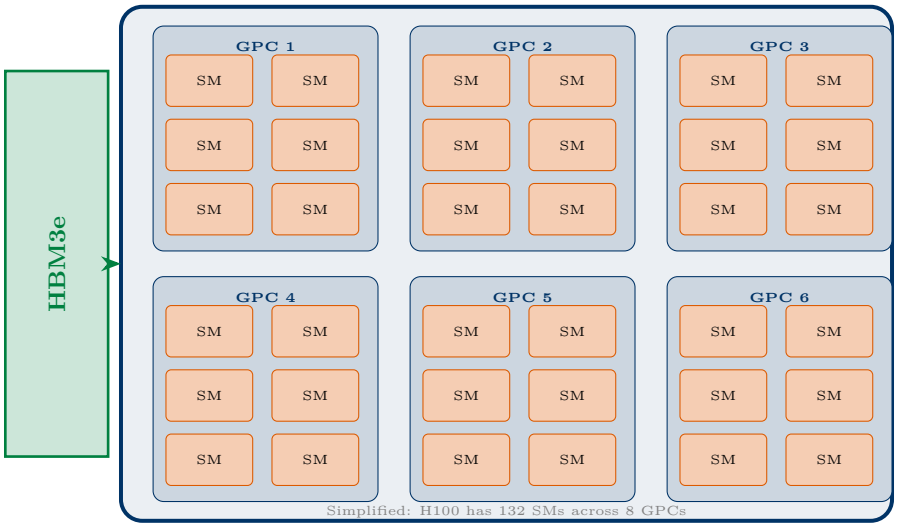
Imagine you need to add the numbers on 10,000 pieces of paper.
A **CPU** is like one expert accountant. Fast, smart, capable of handling complex calculations. But it works through papers one at a time — or maybe a few at a time with multiple cores.
A **GPU** is like 10,000 students with calculators, each working on one paper simultaneously. Each student is slower and less capable than the expert. But together, they finish the whole pile in the time the expert handles one.
For AI, which involves performing the same multiplication millions of times across different data, the 10,000 students win — by orders of magnitude.

2.2 GPU Hardware Architecture

2.2.1 Streaming Multiprocessors (SMs)

The basic computational unit of a GPU is the **Streaming Multiprocessor** (SM). An NVIDIA H100 GPU has 132 SMs. Each SM contains:

- **CUDA cores:** General-purpose floating-point and integer compute units
- **Tensor Cores:** Specialized matrix-multiply units (described below)
- **Registers:** Tiny, ultra-fast memory local to each thread
- **Shared Memory / L1 Cache:** Fast memory shared across threads in one SM
- **Warp Schedulers:** Hardware that manages thread execution



2.2.2 The GPU Memory Hierarchy

Memory in a GPU is organized in layers, from fastest (but tiny) to slowest (but huge):

Memory Type	Size	Bandwidth	Purpose
Registers	256 KB/SM	∼20 TB/s	Per-thread data
L1/Shared Memory	228 KB/SM	∼19 TB/s	Thread cooperation within SM
L2 Cache	50 MB	∼12 TB/s	Cross-SM caching
HBM3e (VRAM)	80–141 GB	3.35 TB/s	Model weights, KV cache
CPU RAM (via NVLink/PCIe)	TBs	900 GB/s	Offload, overflow

Table 2.1: H100 GPU Memory Hierarchy

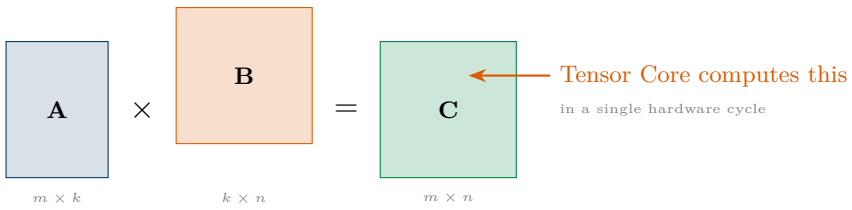
The Golden Rule of GPU Programming

The fastest code is code that keeps data as close to the compute units as possible. If your CUDA kernel reads from HBM when it could read from shared memory, you are leaving 5× performance on the table. Every serious GPU optimization is, at its heart, a memory access optimization.

2.2.3 Tensor Cores: The Matrix Multiplication Engines

Tensor Cores are specialized hardware units inside each SM designed for one specific task: ultra-fast matrix multiplication. They were introduced in the Volta architecture (2017) and have been enhanced in every generation since.

Why does matrix multiplication matter for AI? Because the core operation in every neural network layer is a **matrix multiplication** (GEMM — General Matrix Multiply). When you multiply an input vector by a weight matrix to produce an output, that is a matrix multiply. Modern LLMs perform this operation hundreds of times per forward pass.



The H100 Tensor Core operates on **FP8** precision natively, performing a $16 \times 8 \times 16$ matrix multiplication in a single instruction — the equivalent of 2048 floating-point multiply-add operations per cycle.

GPU Generations and Precision Support

GPU	Arch.	Year	Peak (FP8)	Key Features
A100	Ampere	2020	624 TFLOPS	First FP16 Tensor Cores, NVLink 3
H100 SXM	Hopper	2022	3958 TFLOPS	FP8 native, Transformer Engine, NVLink 4
H200 SXM	Hopper+	2024	3958 TFLOPS	141 GB HBM3e memory
B200	Blackwell	2025	9000 TFLOPS	FP4, 5th-gen Tensor Cores, 30× H100 inference
GB200 NVL72	Blackwell	2025	∼1 EFLOPS	72 GPUs in a rack, disaggregated serving

Table 2.2: NVIDIA GPU Generations for AI Inference

Blackwell's Leap

The NVIDIA B200 (Blackwell) delivers up to 30× the inference performance of the H100 on specific workloads, particularly for large models using FP4 quantization. The fifth-generation Tensor Cores support FP32, FP16, BF16, FP8, INT8, FP6, and FP4 — enabling extremely flexible precision choices.

2.3 CUDA Programming Model

CUDA (Compute Unified Device Architecture) is NVIDIA's programming framework for writing code that runs on GPUs. Every inference runtime — vLLM, TensorRT, PyTorch — ultimately executes CUDA code under the hood.

2.3.1 The Thread Hierarchy

CUDA organizes GPU execution in a strict three-level hierarchy:

- **Thread:** The smallest unit. One thread performs one computation.
- **Block:** A group of up to 1024 threads that share memory and can synchronize.
- **Grid:** A collection of blocks that together run one kernel.

Analogy: School Analogy

Think of a grid as a **school**. Each block is a **classroom**. Each thread is a **student**.
Students in one classroom can pass notes (shared memory). Students in different classrooms cannot talk directly. The principal (GPU scheduler) tells each classroom what lesson to do (*kernel*).

2.3.2 A Simple CUDA Kernel

```
// This function runs on the GPU -- one thread per element
__global__ void vector_add(float* A, float* B, float* C, int N) {
    // Each thread computes its own index
    // blockIdx.x  = which block am I in?
```

```
    // blockDim.x  = how many threads per block?
    // threadIdx.x = which thread am I within my block?
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // Boundary check: don't go past the array end
    if (idx < N) {
        C[idx] = A[idx] + B[idx];   // Each thread adds one element
    }
}

// Called from the CPU (host)
void run_vector_add(float* A, float* B, float* C, int N) {
    int threads_per_block = 256;
    int num_blocks = (N + threads_per_block - 1) / threads_per_block;

    // <<< grid_size, block_size >>> launches the kernel
    vector_add<<<num_blocks, threads_per_block>>>(A, B, C, N);
}
```

Listing 2.1: A simple CUDA vector addition kernel

2.3.3 Memory Management in CUDA

```
// Allocate memory on CPU (host)
float* h_A = (float*)malloc(N * sizeof(float));

// Allocate memory on GPU (device)
float* d_A;
cudaMalloc(&d_A, N * sizeof(float));    // GPU memory allocation

// Transfer data CPU -> GPU
cudaMemcpy(d_A, h_A, N*sizeof(float), cudaMemcpyHostToDevice);

// Run kernel on GPU
my_kernel<<<grid, block>>>(d_A, ...);

// Transfer results GPU -> CPU
cudaMemcpy(h_result, d_result, N*sizeof(float), cudaMemcpyDeviceToHost);

// Free GPU memory
cudaFree(d_A);
```

Listing 2.2: CUDA memory allocation and transfer

Memory Transfer is Expensive

Moving data between CPU and GPU via PCIe costs time — typically 16 GB/s on a PCIe 4.0 ×16 connection. This is 100× slower than reading from GPU HBM. In production inference, the goal is to keep *everything* on the GPU and minimize CPU-GPU transfers.

2.3.4 Warps: The Real Execution Unit

While blocks are the logical unit, hardware executes threads in groups of 32 called **warps**. All 32 threads in a warp execute the same instruction simultaneously. If threads in a warp take different code paths (e.g., an if/else), both paths execute sequentially with some threads masked off. This is called **warp divergence** and is a major source of GPU performance loss.

Exercise 2.1: Thread Index Calculation

A CUDA kernel is launched with a grid of 16 blocks, each containing 256 threads.

1. What is the total number of threads?
2. If `blockIdx.x = 5` and `threadIdx.x = 100`, what is the global thread index?

3. If processing an array of 3000 elements, how many threads will be out of bounds and need a guard check?

2.4 FlashAttention: The Kernel That Changed Inference

Before we leave GPU basics, let's preview one of the most important CUDA optimizations ever invented for AI: **FlashAttention**.

The standard attention computation in a Transformer reads the key, query, and value matrices from HBM, computes a large intermediate matrix (the attention scores), writes it back to HBM, then reads it again to apply softmax. This is **extremely memory-bandwidth-bound**.

FlashAttention (Dao et al., 2022) reorganizes this computation so that all intermediate results stay in shared memory. It never writes the large intermediate matrix to HBM at all. This single change achieves:

- $2 - 4\times$ faster attention on GPU
- $5 - 20\times$ less memory for attention
- Exact same mathematical result as standard attention

FlashAttention is now standard in every production inference engine.

Chapter Summary

GPUs accelerate AI because they perform thousands of operations in parallel. The memory hierarchy (registers $\rightarrow$ shared memory $\rightarrow$ HBM) is central to performance: keep data close to compute. CUDA organizes execution as grids of blocks of threads. Tensor Cores handle matrix multiplication at extreme speed. Modern H100 and Blackwell GPUs support FP8/FP4 precision for even higher throughput. FlashAttention shows how smart kernel design can dramatically improve both speed and memory use.

CHAPTER 3

Transformer Inference Mechanics

Learning Objectives

- Understand how Transformer models generate text token by token
- Learn the two phases of inference: prefill and decode
- Understand why decode is memory-bandwidth-bound, not compute-bound
- Grasp the roofline model for compute vs. memory bound analysis

3.1 How Transformers Generate Text

Large Language Models generate text **one token at a time**. A **token** is roughly a word-piece — "inference" might be one token, while "transformers" might be two: "transform" and "ers".

When you send a prompt to an LLM, here is what happens:

1. Your prompt is tokenized into a list of integer token IDs.
2. The model processes all prompt tokens simultaneously (**prefill phase**).
3. The model generates one output token, then another, then another, until it produces an end-of-sequence token or reaches a length limit (**decode phase**).

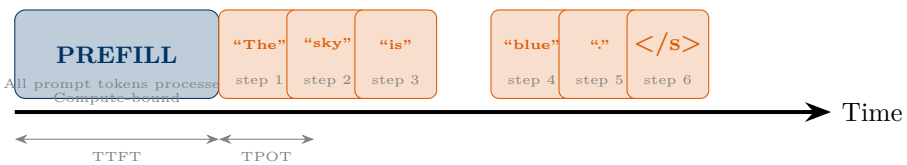
Analogy: The Sentence Completion Game

Imagine you are playing a word-completion game. You read the entire sentence so far, then you predict the most likely next word. You write it down. Now you read the whole sentence again, including the word you just wrote, and predict the next word. Repeat.
This is exactly how an LLM works. The entire conversation history is reread every time a new token is generated. This is called *autoregressive* generation.

3.1.1 The Two Phases in Detail

Prefill processes the entire input prompt in a single forward pass. If your prompt has 512 tokens, all 512 are processed at once, using the GPU's full parallel compute power. Prefill is **compute-bound**: the GPU is busy multiplying matrices.

Decode generates one token per step. At each step, only a single new token is processed (the most recently generated one), while all previous tokens are accessible through the KV cache (Chapter 4). Decode is **memory-bandwidth-bound**: the GPU spends most of its time loading model weights from HBM, not computing.



3.2 The Roofline Model: Compute vs. Memory Bound

To understand why decode is slow, we need the **roofline model** — a simple framework for determining whether a computation is limited by arithmetic speed or memory bandwidth.

Every GPU has two fundamental limits:

- **Peak FLOPS:** How many floating-point operations per second it can perform (H100: ~1979 TFLOPS in FP16)
- **Peak Memory Bandwidth:** How fast it can read data from HBM (H100: 3.35 TB/s)

For a given computation, the **arithmetic intensity** is:

$$\text{Arithmetic Intensity} = \frac{\text{FLOP performed}}{\text{bytes read from memory}}$$

If arithmetic intensity is **above** a threshold (called the ridge point), the operation is **compute-bound**: your GPU is fast enough but you're not feeding it enough data.

If arithmetic intensity is **below** the ridge point, the operation is **memory-bandwidth-bound**: your GPU is waiting for data from memory.

For decode with a single request:

- We read the entire model (~140 GB for a 70B parameter FP16 model)
- We perform only ~280 GFLOPs of computation per token
- Arithmetic intensity $\approx$ 2 FLOP/byte — far below the H100's ridge point of ~590 FLOP/byte

The Decode Efficiency Problem

For a single request, the GPU is roughly **99% idle during decode**. We load 140 GB of model weights to produce maybe 50 bytes of output token. This is deeply inefficient — and it's the core reason batching is so important in inference engineering.

3.3 Inside a Transformer Layer

A single Transformer layer contains two main components:

1. **Multi-Head Self-Attention (MHA):** Each token attends to every other token in the sequence.
2. **Feed-Forward Network (FFN):** A two-layer MLP applied independently to each token.

3.3.1 The Attention Computation

For a sequence of n tokens, the attention computation is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

where Q, K, V are the query, key, and value matrices derived from the input by multiplying with learned weight matrices W_Q, W_K, W_V.

During **decode**, we only compute Q for the new token, but we need all previous K and V values. This is why we need a **KV Cache** (Chapter 4).

3.3.2 Memory Footprint of a Large Model

For a Llama-3 70B model in FP16:

Component	Parameters	Memory (FP16)
Embedding layers	0.5B	1 GB
80 Attention layers	40B	80 GB
80 FFN layers	28B	56 GB
Other (LN, head)	1.5B	3 GB
Total	**70B**	**~140 GB**

This model requires **two H100 80GB GPUs** just to load the weights, before any KV cache or batch data.

Chapter Summary

Transformers generate text autoregressively — one token at a time. The prefill phase processes the entire prompt in parallel (compute-bound), while the decode phase generates one token at a time (memory-bandwidth-bound). The roofline model helps understand why: during decode we load all model weights for minimal computation. Batching multiple requests together is the key to making decode efficient. Large models need substantial GPU memory just for their weights.

CHAPTER 4

KV Cache and Memory Management

Learning Objectives

- Understand why the KV cache exists and what it stores
- Learn how PagedAttention eliminates KV cache fragmentation
- Calculate KV cache memory requirements for production systems
- Understand prefix caching and its performance benefits

4.1 Why We Need a KV Cache

Recall the decode phase: we generate one new token at a time, but each token requires attention over *all* previous tokens.

Without caching: at step t, to generate token $t+1$, we would recompute all the key and value vectors for tokens 1 through t. This means the computation grows linearly with sequence length: generating a 1000-token response costs $1+2+3+\ldots+1000 = 500{,}500$ attention computations instead of just 1000.

The **KV Cache** solves this by **storing** the key and value matrices computed for every past token. When generating token $t+1$, we reuse all stored K and V from steps 1 through t and only compute new K and V for the latest token.

Analogy: The Meeting Notes

Imagine you are taking notes in a long meeting. Instead of re-reading the entire transcript every time you want to write the next sentence, you keep a notepad of the key points discussed so far.
Each time you need to reference the past, you glance at your notepad — not the entire transcript.
The KV cache is that notepad. It stores the important "memory" of past tokens so the GPU doesn't recompute it every step.

4.2 KV Cache Memory Calculation

The KV cache size for a single sequence is:

$$\text{KV cache size} = 2 \times n_{\text{layers}} \times n_{\text{heads}} \times d_{\text{head}} \times \text{seq\_len} \times \text{bytes\_per\_element}$$

The factor 2 accounts for both keys (K) and values (V).

For Llama-3 70B (FP16, 4096 sequence length):

$$2 \times 80 \times 64 \times 128 \times 4096 \times 2 \approx \mathbf{10.7}\ \text{GB}$$

Memory Shock

A single conversation of 4096 tokens with Llama-3 70B uses **10.7 GB** of KV cache — on top of the 140 GB for weights. Serving 10 simultaneous users requires 107 GB of KV cache alone. Efficient KV cache management is one of the biggest challenges in production inference.

4.3 The Problem: KV Cache Fragmentation

Before PagedAttention, every inference engine allocated KV cache memory using a single large contiguous block per sequence.

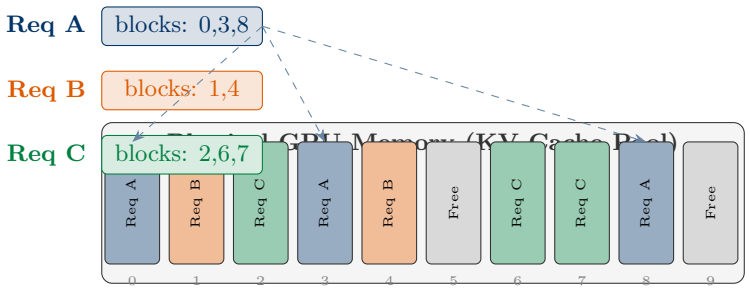
Imagine you are running a hotel. Every guest (request) is allocated a room block: a contiguous range of rooms. But you don't know in advance how long each guest will stay. You must reserve the maximum possible stay (max sequence length) for each guest upfront. The result: many rooms sit empty, but no new guests can be booked because there's no single contiguous block large enough.

In production, this **fragmentation** means that even when 30% of GPU memory is technically free, it cannot be used because it is scattered in small unusable pieces. Studies found that before PagedAttention, 60–80% of GPU memory allocated for KV caches was wasted.

4.4 PagedAttention: Solving Fragmentation

PagedAttention (Kwon et al., SOSP 2023 — the core innovation behind vLLM) borrows an idea from operating system virtual memory: **paging**.

Instead of one contiguous block per request, memory is divided into fixed-size **blocks** (typically 16 or 32 tokens per block). A request uses a **block table** — a list of pointers to the physical blocks it owns. Blocks do not need to be contiguous in memory.



PagedAttention Impact

PagedAttention reduces KV cache waste from 60–80% down to under 5%. vLLM achieves 14–24× higher throughput than naive HuggingFace Transformers and 2.2–3.5× higher than early Text Generation Inference (TGI) engines for LLaMA models on NVIDIA GPUs.

4.5 Prefix Caching

Many queries share common prefixes: the same system prompt, the same document being analyzed, the same few-shot examples.

Prefix caching (also called *prompt caching* or *RadixAttention* in SGLang) stores the KV cache of common prefixes and **reuses** them across multiple requests.

If 1000 users all send queries starting with the same 2000-token system prompt, prefix caching means the KV computation for those 2000 tokens is performed *once*, not 1000 times.

```
from vllm import LLM, SamplingParams

# Initialize vLLM with prefix caching enabled
llm = LLM(
    model="meta-llama/Llama-3-70b-instruct",
    enable_prefix_caching=True,    # Enable prefix caching
    gpu_memory_utilization=0.90,  # Use 90% of GPU memory for KV pool
    tensor_parallel_size=2,       # Across 2 GPUs
)
```

```
# Many requests sharing the same system prompt will be served
# faster because the KV cache for the system prompt is reused
system_prompt = "You are a helpful coding assistant. " * 200  # long prompt

responses = llm.generate(
    [system_prompt + user_q for user_q in user_questions],
    SamplingParams(temperature=0.7, max_tokens=512)
)
```

Listing 4.1: Enabling prefix caching in vLLM

4.6 KV Cache Quantization

Since KV cache can consume more memory than the model weights themselves for long sequences, quantizing the KV cache is a powerful optimization.

Recent research shows that INT8 quantization of the KV cache achieves a **4× memory reduction** with reconstruction error below 0.004 and attention score error below 0.1 even for 8K-dimensional heads. FP8 KV quantization is now natively supported in vLLM and TensorRT-LLM.

```
from vllm import LLM

llm = LLM(
    model="meta-llama/Llama-3-70b-instruct",
    kv_cache_dtype="fp8",          # Use FP8 for KV cache
    enable_prefix_caching=True,
    gpu_memory_utilization=0.92,
)
# FP8 KV cache reduces memory by ~2x vs FP16
# Minimal accuracy degradation in practice
```

Listing 4.2: FP8 KV cache quantization in vLLM

Chapter Summary

The KV cache stores computed keys and values for all past tokens, allowing efficient autoregressive generation without recomputation. KV cache memory grows linearly with sequence length and can easily exceed model weight memory. PagedAttention solves fragmentation by using block-based allocation similar to OS virtual memory, enabling near-zero waste. Prefix caching reuses KV state for shared prompt prefixes. FP8/INT8 quantization of the KV cache reduces memory by 2–4× with minimal accuracy impact.

CHAPTER 5

Quantization: Fitting More Into Less Memory

Learning Objectives

- Understand what quantization is and why it matters for inference
- Learn the difference between INT8, FP8, INT4, FP4 quantization
- Understand post-training quantization (PTQ) vs. quantization-aware training
- Apply AWQ and GPTQ quantization in practice
- Understand the accuracy–speed–memory tradeoffs

5.1 What Is Quantization?

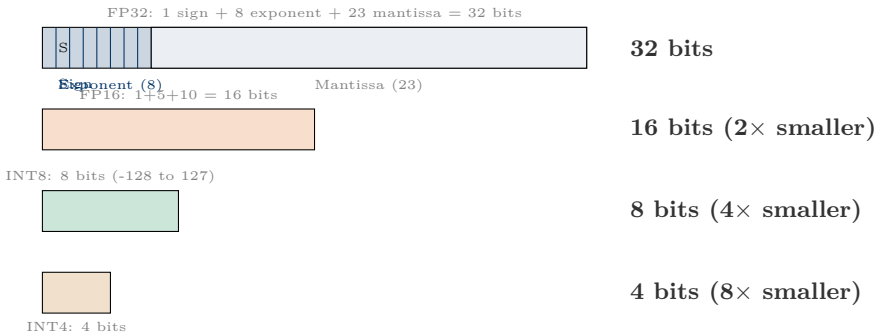
Neural network weights are typically stored in full precision: **FP32** (32 bits per number) or **FP16/BF16** (16 bits per number).

Quantization replaces these high-precision numbers with lower-precision representations — fewer bits per number — that are **good enough** for inference.

Analogy: The Photo Compression Analogy

Imagine a photograph stored at full resolution: 48 megapixels, 24 bits per pixel. The file is huge — say, 200 MB.
Now you apply JPEG compression. The image now takes 2 MB. When you look at it on your phone screen, you cannot tell the difference.
Quantization is compression for AI model weights. You reduce the number of bits per weight. The model takes less memory, runs faster, and (if done carefully) produces answers that are almost identical to the original.

5.2 Number Formats: A Visual Guide



Format	Bits	Relative Size	Dynamic Range	Use Case
FP32	32	1×	1.2×10^{-38} to 3.4×10^{38}	Training reference
BF16	16	2×	Same range as FP32	Default training/inference

Format	Bits	Relative Size	Dynamic Range	Use Case
FP16	16	2×	6×10^{-5} to 65504	Common inference
FP8 (E4M3)	8	4×	−448 to 448	H100/Blackwell inference
INT8	8	4×	−128 to 127	Weight/activation quantization
INT4/FP4	4	8×	−8 to 7	Aggressive compression

Table 5.1: Number Formats for AI Inference

5.3 Post-Training Quantization Methods

Post-Training Quantization (PTQ) applies quantization to an already trained model, without any additional training.

5.3.1 GPTQ: Round-to-Nearest with Correction

GPTQ (Frantar et al., 2022) is a popular INT4 quantization method that:

1. Quantizes weights layer by layer, in order
2. After quantizing each weight, adjusts remaining weights to compensate for the error introduced
3. Uses a small calibration dataset to guide quantization

```python
from transformers import AutoModelForCausalLM, AutoTokenizer
from optimum.gptq import GPTQQuantizer

# Load a pre-quantized GPTQ model (4-bit)
# Models are available on HuggingFace Hub with "-GPTQ" suffix
model = AutoModelForCausalLM.from_pretrained(
    "TheBloke/Llama-2-70B-GPTQ",  # Pre-quantized 4-bit model
    device_map="auto",            # Auto-distribute across GPUs
    torch_dtype="auto",
)
# This 70B model now fits in ~35GB instead of ~140GB
```

Listing 5.1: Loading a GPTQ quantized model

5.3.2 AWQ: Activation-Aware Weight Quantization

AWQ (Lin et al., 2023) is a smarter quantization technique that observes which weights are most **important** for accuracy (by analyzing activations) and protects them from quantization error.

Key insight: *not all weights are equally important.* About 1% of weights are "salient" — they have large activations and dominate the model's behavior. AWQ keeps these weights at higher precision while aggressively quantizing the rest.

```python
from vllm import LLM, SamplingParams

# Load AWQ 4-bit quantized model
llm = LLM(
    model="casperhansen/llama-3-70b-instruct-awq",
    quantization="awq",           # Tell vLLM this is an AWQ model
    tensor_parallel_size=2,       # Still need 2 GPUs for 4-bit 70B
    gpu_memory_utilization=0.92,
)
```

```
output = llm.generate(
    ["Explain transformer attention in one sentence."],
    SamplingParams(temperature=0.7, max_tokens=100)
)
print(output[0].outputs[0].text)
```

Listing 5.2: Using vLLM with AWQ quantized model

5.3.3 FP8 Quantization on H100/Blackwell

The H100 introduced native FP8 hardware support through its Transformer Engine. FP8 requires minimal post-processing compared to INT quantization because it maintains a floating-point representation, preserving the dynamic range needed by neural network activations.

```
from vllm import LLM

# Enable FP8 quantization (requires H100/Blackwell or newer)
llm = LLM(
    model="meta-llama/Llama-3-70b-instruct",
    quantization="fp8",           # Use FP8 for both weights and activations
    gpu_memory_utilization=0.90,
    tensor_parallel_size=1,       # 70B FP8 fits on single H100!
)
```

Listing 5.3: FP8 inference with vLLM

5.4 Quantization Accuracy Tradeoffs

Method	Bits	Memory	Speed	Accuracy Loss
BF16 (baseline)	16	140 GB	1.0×	0%
FP8	8	70 GB	1.9×	<0.1%
INT8 (SmoothQuant)	8	70 GB	1.7×	<0.5%
INT4 (AWQ)	4	35 GB	2.5×	1–3%
INT4 (GPTQ)	4	35 GB	2.3×	1–5%
FP4 (Blackwell)	4	35 GB	4.0×	<1%

All numbers are approximate for Llama-3 70B on H100. Accuracy loss measured as perplexity increase on common benchmarks.

Which Quantization to Choose?

- **H100/Blackwell, maximum speed:** FP8 (native hardware support, minimal accuracy loss)
- **Memory constrained, quality matters:** AWQ INT4 (best accuracy at 4-bit)
- **Maximum compression:** GPTQ INT4 with group size 32
- **Edge/CPU deployment:** GGUF with llama.cpp (Q4_K_M is excellent)

Chapter Summary

Quantization reduces model weight precision to save memory and increase speed. Lower precision (FP8, INT4) means smaller models, higher throughput, but some accuracy loss. GPTQ and AWQ are leading 4-bit post-training quantization methods. FP8 is now natively supported on H100 and Blackwell GPUs with negligible accuracy loss. On Blackwell, FP4 delivers 4× the throughput of H100 FP8. Always benchmark accuracy on your specific task before deploying a quantized model.

Speculative Decoding

Learning Objectives

- Understand why speculative decoding speeds up inference
- Learn the draft-verify paradigm
- Understand acceptance rates and when speculation helps
- Apply speculative decoding with vLLM and TensorRT-LLM

6.1 The Bottleneck: Sequential Decode Steps

Recall that decode generates one token at a time. Each step requires a full forward pass through the model. Even if you have a very fast GPU, you cannot parallelize these steps: token $n+1$ cannot be generated until token n is known.

Analogy: The Draft and Reviewer

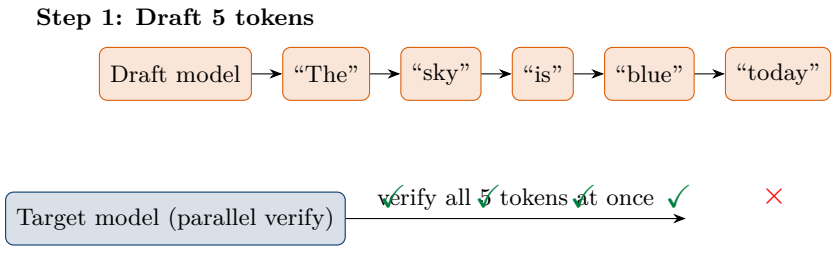
Imagine a document that must be approved by an expert (the large model). The expert is slow but always right.
You hire a fast intern (the small model) to write a draft. The intern quickly suggests the next 5 words. Then the expert reviews all 5 words in a single pass — accepting most of them, and correcting any mistakes.
Result: the expert does one review instead of five. If the intern is correct 4 out of 5 times, the overall speed roughly doubles.
This is *speculative decoding*.

6.2 How Speculative Decoding Works

Speculative decoding uses two models:

- **Draft model:** A small, fast model that quickly generates k candidate tokens.
- **Target model:** The large, accurate model that verifies all k tokens in **one parallel forward pass**.

The key insight is that the target model can verify k tokens in parallel (similar to prefill) rather than generating them sequentially. If the draft is mostly correct, we get multiple output tokens per target model forward pass.



6.2.1 N-gram and EAGLE Speculation

Beyond a separate small model, modern systems use:

- **N-gram speculation:** Predict next tokens based on n-gram statistics from the current context. Fast and requires no extra model.
- **EAGLE/EAGLE-3:** Uses a shallow draft head that shares the target model's hidden states. Achieves 3× speedup on typical text.
- **Medusa:** Multiple parallel draft heads on the same model backbone.

```python
from vllm import LLM, SamplingParams

# Method 1: Use a small draft model
llm = LLM(
    model="meta-llama/Llama-3-70b-instruct",        # Target model
    speculative_model="meta-llama/Llama-3-8b-instruct",  # Draft model
    num_speculative_tokens=5,   # Generate 5 draft tokens per step
    tensor_parallel_size=2,
)

# Method 2: N-gram based speculation (no extra model needed)
llm_ngram = LLM(
    model="meta-llama/Llama-3-70b-instruct",
    speculative_model="[ngram]",     # Built-in n-gram drafter
    ngram_prompt_lookup_max=4,       # Look back 4 tokens for patterns
    num_speculative_tokens=5,
    tensor_parallel_size=2,
)

result = llm.generate(
    ["Write a Python function to sort a list:"],
    SamplingParams(temperature=0.0, max_tokens=200)  # Greedy is ideal for speculation
)
```

Listing 6.1: Speculative decoding with vLLM

When Speculation Helps Most

Speculative decoding helps most when:

- Output is predictable (code, structured text, repetitive responses)
- Batch size is small (low utilization otherwise)
- Latency matters more than throughput

Speculative decoding *hurts* throughput at high batch sizes because it adds verification overhead. At a batch size of 32+, continuous batching is usually better than speculation.

Chapter Summary

Speculative decoding uses a fast draft model to propose multiple tokens, which the target model verifies in parallel. When the draft is mostly correct, this delivers 2–4× lower latency for single-request serving. EAGLE-3 and n-gram speculation are the preferred methods in 2025. Speculation is most beneficial for low-batch, latency-sensitive use cases, not high-throughput batch serving.

CHAPTER 7

Inference Runtimes: vLLM, TensorRT-LLM, and SGLang

Learning Objectives

- Understand the architecture of the three leading inference runtimes
- Compare vLLM, TensorRT-LLM, and SGLang across key dimensions
- Know which runtime to choose for a given use case
- Deploy a model with vLLM and a REST API

7.1 The Inference Runtime Stack

An **inference runtime** is the software layer that takes a trained model and serves it efficiently to users. Think of it as the kitchen of a restaurant: the model is the recipe, and the runtime is the kitchen with all its equipment, scheduling, and organization.

Every production runtime must handle:

- **Model loading:** Reading weights from disk, placing on GPUs
- **Request queuing:** Accepting HTTP/gRPC requests from users
- **Batching:** Grouping multiple requests together for efficiency
- **Scheduling:** Deciding which requests to process next
- **Memory management:** Allocating/freeing KV cache for each request
- **Output:** Streaming tokens back to users as they are generated

7.2 vLLM: The Production Standard

vLLM (UC Berkeley, 2023, now a PyTorch Foundation project) is the most widely adopted open-source inference engine. Its core innovations:

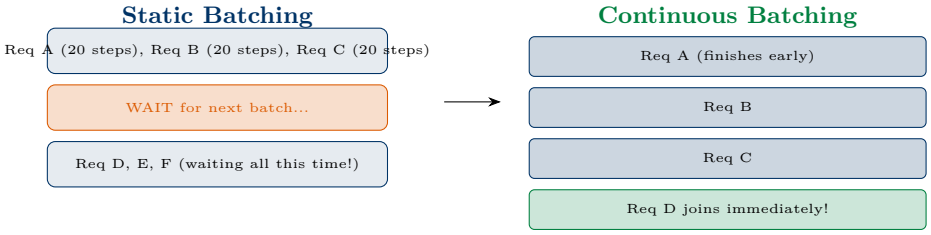
- **PagedAttention:** Eliminates KV cache fragmentation (Chapter 4)
- **Continuous batching:** New requests join as soon as a slot frees up
- **Prefix caching:** Reuses KV state for shared prefixes
- **OpenAI-compatible API:** Drop-in replacement for OpenAI API

Typical performance: 120–160 requests/second, 50–80ms TTFT with 100 concurrent users for Llama-class models on H100.

7.2.1 Continuous Batching

Traditional batching waits for a batch to be full before starting. This is like a bus that only departs when every seat is full — efficient for the bus, but terrible for passengers who arrived early.

Continuous batching (also called iteration-level scheduling) allows new requests to join the batch at every decode step. When request A finishes at step 50, a new request immediately fills that slot.



7.2.2 Running vLLM

```
# Install vLLM
pip install vllm

# Start the server (OpenAI-compatible API on port 8000)
python -m vllm.entrypoints.openai.api_server \
    --model meta-llama/Llama-3-70b-instruct \
    --tensor-parallel-size 2 \          # Use 2 GPUs
    --gpu-memory-utilization 0.90 \     # Use 90% of GPU memory for KV cache
    --enable-prefix-caching \           # Enable prefix caching
    --kv-cache-dtype fp8 \              # FP8 KV cache
    --max-num-seqs 256 \                # Max concurrent sequences
    --port 8000
```

Listing 7.1: Starting the vLLM OpenAI-compatible server

```
import openai

# vLLM is OpenAI API compatible
client = openai.OpenAI(
    base_url="http://localhost:8000/v1",
    api_key="not-needed"                # vLLM doesn't require a key
)

response = client.chat.completions.create(
    model="meta-llama/Llama-3-70b-instruct",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Explain KV cache in 2 sentences."}
    ],
    max_tokens=200,
    stream=True                         # Enable streaming
)

for chunk in response:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="", flush=True)
```

Listing 7.2: Querying the vLLM server

7.3 TensorRT-LLM: Maximum NVIDIA Performance

TensorRT-LLM (NVIDIA) compiles a model into a highly optimized engine specifically for a particular GPU, precision, and sequence configuration.

Key advantages:

- Lowest latency at low concurrency (35–50ms TTFT vs. vLLM's 50–80ms)
- Deep GPU kernel fusion (multiple operations merged into one kernel)
- FP8/FP4 native support for Blackwell
- Disaggregated serving via NVIDIA Dynamo

Key disadvantages:

- Complex engine compilation (can take hours per model)
- NVIDIA-only (no AMD, Intel)
- Requires re-compilation when model or GPU changes

```
# Install TensorRT-LLM
pip install tensorrt_llm

# Step 1: Convert model to TensorRT-LLM checkpoint format
python convert_checkpoint.py \
    --model_dir ./llama-3-70b \
    --output_dir ./trt-checkpoint \
    --dtype float16

# Step 2: Build the TRT engine (this compiles for your specific GPU)
trtllm-build \
    --checkpoint_dir ./trt-checkpoint \
    --output_dir ./trt-engine \
    --max_input_len 2048 \
    --max_output_len 512 \
    --max_batch_size 32 \
    --tp_size 2                  # Tensor parallel across 2 GPUs

# Step 3: Run inference
python run.py \
    --engine_dir ./trt-engine \
    --max_output_len 100 \
    --input_text "Explain GPU memory hierarchy:"
```

Listing 7.3: Building and running a TensorRT-LLM engine

7.4 SGLang: Structured Generation Language

SGLang (Zheng et al., 2024) introduces *RadixAttention* — an automatic KV cache reuse system that builds a radix tree of all cached prefixes and shares them optimally across requests.

SGLang excels for:

- Long-document processing (PDF analysis, RAG)
- Conversation-style traffic with long shared history
- Multi-turn agents that reuse the same system context
- Structured generation (JSON output, grammars)

```
pip install sglang

# Launch SGLang server
python -m sglang.launch_server \
    --model-path meta-llama/Llama-3-70b-instruct \
```

```
    --tp 2 \                    # Tensor parallel size
    --mem-fraction-static 0.82 \
    --enable-flashinfer         # Use FlashInfer attention backend
```

Listing 7.4: Starting SGLang server

7.5 Runtime Comparison: The Decision Guide

Criterion	vLLM	TRT-LLM	SGLang
Setup time	Minutes	Hours/days	Minutes
TTFT (low concurrency)	50–80ms	35–50ms	50–80ms
Throughput (high concurrency)	Excellent	Excellent	Excellent
Hardware	Any (NVIDIA/AMD)	NVIDIA only	NVIDIA/AMD
Prefix caching	Good	Good	Excellent (RadixAttention)
Model support	Widest	NVIDIA-curated	Growing
API compatibility	OpenAI	Custom/Triton	OpenAI
Best for	General production	Max NVIDIA perf	Long context/RAG

Table 7.1: Inference Runtime Comparison (2025)

Chapter Summary

vLLM is the go-to for most production deployments: easy setup, OpenAI-compatible, excellent throughput. TensorRT-LLM achieves the lowest latency on NVIDIA hardware but requires compilation effort. SGLang leads for long-context workloads with RadixAttention. All three support modern features: FP8, continuous batching, prefix caching. Choose based on your traffic shape, ops maturity, and latency SLAs.

CHAPTER 8

Distributed Inference: Parallelism Strategies

Learning Objectives

- Understand why a single GPU is often not enough
- Learn tensor parallelism, pipeline parallelism, and sequence parallelism
- Understand NVLink and its role in multi-GPU scaling
- Apply tensor parallelism with vLLM

8.1 When One GPU Is Not Enough

A Llama-3 70B model in FP16 requires 140 GB of memory — far more than a single H100's 80 GB. Even smaller models may benefit from multiple GPUs for throughput.

Distributing a model across multiple GPUs requires a **parallelism strategy**. There are three main approaches:

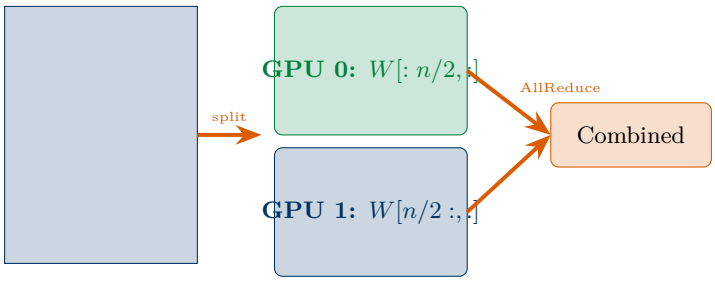
8.2 Tensor Parallelism

Tensor parallelism (TP) splits individual matrices *across* GPUs.

Analogy: The Heavy Book

Imagine a very large book that is too heavy for one person to carry. Instead of one person carrying the whole book, two people each carry half — one carries pages 1–500, the other carries pages 501–1000.
Tensor parallelism works similarly. A large weight matrix (say, 8192×8192) is split across 2 GPUs: each GPU stores a 4096×8192 slice.
Both GPUs compute in parallel, then combine their partial results. The communication (combining results) uses NVLink for very high bandwidth.

With TP-4 (4 GPUs), each GPU holds 1/4 of each weight matrix. Each forward pass requires an **AllReduce** communication operation to combine results across all 4 GPUs.



8.2.1 Launching Tensor-Parallel vLLM

```
# 70B model across 2 H100s with TP=2
python -m vllm.entrypoints.openai.api_server \
    --model meta-llama/Llama-3-70b-instruct \
    --tensor-parallel-size 2        # Split across 2 GPUs
```

```
# 405B model across 8 H100s with TP=8
python -m vllm.entrypoints.openai.api_server \
    --model meta-llama/Llama-3-405b-instruct \
    --tensor-parallel-size 8        # Split across 8 GPUs
```

Listing 8.1: Running vLLM with tensor parallelism

8.3 Pipeline Parallelism

Pipeline parallelism (PP) assigns different model *layers* to different GPUs. Layers 0–19 run on GPU 0, layers 20–39 run on GPU 1, etc.

Analogy: The Assembly Line

In a car factory, one station installs the engine, another the doors, another the paint. Each station works on a different car simultaneously.
In pipeline parallelism, GPU 0 finishes processing layers 0–19 for batch 1 and passes the intermediate activations to GPU 1. While GPU 1 works on batch 1, GPU 0 starts processing batch 2.
The pipeline keeps all GPUs busy, like a factory assembly line.

Pipeline parallelism communicates via a simpler point-to-point activation transfer (not AllReduce), making it more efficient across slower interconnects (e.g., InfiniBand, PCIe) than tensor parallelism.

8.4 NVLink: The Fast Lane Between GPUs

Communication between GPUs is the bottleneck in distributed inference. NVLink is NVIDIA's high-speed GPU interconnect, far faster than PCIe:

Interconnect	Bandwidth (each dir.)	Use Case
PCIe 5.0 ×16	64 GB/s	CPU–GPU
NVLink 4 (H100)	900 GB/s	GPU–GPU
NVLink 5 (B200)	1800 GB/s	GPU–GPU
NVSwitch (GB200 NVL72)	57.6 TB/s total	Rack-scale

With NVLink 4 on a DGX H100 system (8 GPUs), the AllReduce bandwidth for tensor parallelism is **28**× faster than a PCIe-only configuration. This is why TP works well intra-node but poorly across nodes.

The Parallelism Recipe

- **TP within a node** (GPUs on same server, NVLink connected)
- **PP across nodes** (different servers, InfiniBand connected)
- **TP=8, PP=N/8 for very large models** (405B+)

8.5 Disaggregated Prefill and Decode

An emerging production pattern separates the prefill and decode phases onto **different hardware**:

- **Prefill GPUs:** Optimized for high compute (processing long prompts)
- **Decode GPUs:** Optimized for memory bandwidth (fast token generation)

NVIDIA Dynamo (announced GTC 2025) is an orchestration layer that implements this pattern on top of vLLM, TensorRT-LLM, or SGLang. By routing prefill and decode to specialized hardware, it can significantly reduce queue depth and improve both TTFT and throughput simultaneously.

Chapter Summary

Large models require multi-GPU deployment. Tensor parallelism splits weight matrices across GPUs — ideal for intra-node with NVLink. Pipeline parallelism assigns layers to GPUs — better for inter-node. NVLink provides the high-bandwidth interconnect that makes tensor parallelism practical. Disaggregated prefill/decode is an emerging frontier that dedicates different hardware to the two phases of inference.

CHAPTER 9

Observability: Monitoring Inference Systems

Learning Objectives

- Understand the key metrics to monitor in a production inference system
- Learn to use NVIDIA profiling tools: nvtop, nsys, ncu
- Set up Prometheus and Grafana for inference monitoring
- Diagnose common performance bottlenecks from metrics

9.1 The Three Pillars of Observability

A production inference system must be observable:

- **Metrics:** Numerical measurements over time (latency p99, GPU util%)
- **Logs:** Events with timestamps (errors, request traces)
- **Traces:** Request-level timing breakdown (prefill time, queue time)

9.2 Key Metrics to Monitor

Metric	What It Measures	Action if Bad
GPU utilization	% time GPU is active	<60%: add batching; increase load
GPU memory %	VRAM in use	>95%: reduce batch size or add GPUs
TTFT (p50/p95/p99)	Latency to first token	High: check queue depth, prefill load
TPOT (p50/p95)	Latency per output token	High: check decode batch size
Throughput (tok/s)	Tokens generated per second	Low: check GPU util, batching
Request queue depth	Requests waiting	>0 for >1s: scale up
KV cache usage %	Cache pool utilization	Near 100%: risk of OOM eviction
Token generation rate	New tokens/s overall	Declining: memory pressure

Table 9.1: Production Inference Monitoring Metrics

9.3 GPU Profiling Tools

9.3.1 nvtop: Real-Time GPU Monitor

```
# Install nvtop
apt-get install nvtop
```

```
# Run nvtop
nvtop
# Shows: GPU util%, memory, power, temperature for all GPUs
# Press F2 to configure display, q to quit
```

Listing 9.1: Using nvtop for GPU monitoring

9.3.2 Nsight Systems: Timeline Profiling

```
# Profile a vLLM inference run
nsys profile \
    --trace cuda,nvtx,osrt \
    --output inference_profile \
    python inference_benchmark.py

# Open in Nsight Systems GUI
nsys-ui inference_profile.nsys-rep
# Shows: kernel timelines, memory transfers, CPU/GPU overlap
```

Listing 9.2: Profiling with Nsight Systems

9.3.3 Nsight Compute: Kernel-Level Analysis

```
# Profile specific CUDA kernels (use on small workloads)
ncu --set full \
    --kernel-name "flash_fwd_kernel" \
    --export kernel_profile \
    python run_attention.py

# Key metrics to look for:
# - SM active cycles: is GPU compute actually busy?
# - Memory throughput: are we hitting bandwidth limits?
# - Warp efficiency: are warps diverging?
```

Listing 9.3: Per-kernel profiling with ncu

9.4 Prometheus and Grafana for Inference

vLLM exposes metrics natively at `/metrics` in Prometheus format:

```
python -m vllm.entrypoints.openai.api_server \
    --model meta-llama/Llama-3-70b-instruct \
    --tensor-parallel-size 2 \
    --port 8000 \
    --metrics-enabled          # Enable Prometheus metrics
# Metrics available at http://localhost:8000/metrics
```

Listing 9.4: Starting vLLM with metrics

```
# prometheus.yml
scrape_configs:
  - job_name: 'vllm'
    static_configs:
      - targets: ['localhost:8000']
    metrics_path: '/metrics'
    scrape_interval: 5s

# Key vLLM metrics:
# vllm:num_requests_running        - active requests
```

```
# vllm:gpu_cache_usage_perc        - KV cache % used
# vllm:e2e_request_latency_seconds - end-to-end latency histogram
# vllm:generation_tokens_total     - tokens generated
```

Listing 9.5: Prometheus scrape config for vLLM

Chapter Summary

Production inference requires continuous monitoring. Key metrics are GPU utilization, KV cache usage, TTFT, TPOT, and throughput. NVIDIA tools (nvtop, nsys, ncu) provide GPU-level insight. vLLM's Prometheus metrics enable Grafana dashboards for operational monitoring. High queue depth means scale up. Low GPU utilization means increase batch size or adjust scheduling.

CHAPTER 10

Production Deployment with Kubernetes

Learning Objectives

- Understand how to containerize and deploy inference servers
- Configure GPU resources in Kubernetes
- Implement auto-scaling based on latency metrics
- Set up health checks and rolling updates for zero-downtime deployments

10.1 Containerizing a vLLM Server

```
# Dockerfile
FROM nvcr.io/nvidia/cuda:12.4.0-devel-ubuntu22.04

# Install Python and dependencies
RUN apt-get update && apt-get install -y python3-pip python3-dev && \
    pip install vllm==0.7.0 huggingface_hub

# Pre-download the model (or mount as a volume in production)
RUN python3 -c "from huggingface_hub import snapshot_download; \
    snapshot_download('meta-llama/Llama-3-8b-instruct')"

# Expose the API port
EXPOSE 8000

# Start the vLLM server
CMD ["python3", "-m", "vllm.entrypoints.openai.api_server", \
     "--model", "meta-llama/Llama-3-8b-instruct", \
     "--port", "8000", \
     "--gpu-memory-utilization", "0.90"]
```

Listing 10.1: Dockerfile for a vLLM inference server

10.2 Kubernetes Deployment

```
# vllm-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm-llama3-8b
  labels:
    app: vllm-inference
spec:
  replicas: 2                  # Start with 2 replicas
  selector:
    matchLabels:
      app: vllm-inference
  template:
    metadata:
      labels:
        app: vllm-inference
    spec:
```

```
      containers:
      - name: vllm
        image: my-registry/vllm-llama3:latest
        ports:
        - containerPort: 8000
        resources:
          limits:
            nvidia.com/gpu: "1"      # 1 GPU per pod
            memory: "40Gi"
            cpu: "8"
          requests:
            nvidia.com/gpu: "1"
            memory: "32Gi"
            cpu: "4"
        env:
        - name: CUDA_VISIBLE_DEVICES
          value: "0"
        livenessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 60
          periodSeconds: 30
        readinessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 90
          periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: vllm-service
spec:
  selector:
    app: vllm-inference
  ports:
  - port: 80
    targetPort: 8000
  type: LoadBalancer
```

Listing 10.2: Kubernetes Deployment for vLLM

10.3 Horizontal Pod Autoscaler

```
# hpa-vllm.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: vllm-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: vllm-llama3-8b
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Pods
    pods:
      metric:
        name: vllm_queue_depth    # Scale when queue is growing
      target:
        type: AverageValue
        averageValue: "5"         # Scale up when avg queue > 5 requests
```

Listing 10.3: HPA based on custom latency metric

```
# Apply deployment
kubectl apply -f vllm-deployment.yaml

# Check pod status
kubectl get pods -l app=vllm-inference

# Check GPU allocation
kubectl describe pod vllm-llama3-8b-<pod-id>

# View logs
kubectl logs -f deployment/vllm-llama3-8b

# Check service
kubectl get service vllm-service
```

Listing 10.4: Deploying to Kubernetes

Chapter Summary

Production inference runs on Kubernetes with GPU-aware pods. Containerize with NVIDIA CUDA base images and expose a health endpoint. Specify `nvidia.com/gpu` resource limits. Use HPA with custom metrics (queue depth, latency) for autoscaling. Set readiness probes with long initial delays — model loading takes time. Use rolling updates for zero-downtime deployments.

PART II

Advanced Topics

CHAPTER 11

Long Context, Multimodal, and Agentic Inference

Learning Objectives

- Understand the unique challenges of long-context inference
- Learn how multimodal models handle images, audio, and video
- Understand agentic inference patterns and their infrastructure needs

11.1 Long-Context Inference

Models with 128K–1M token context windows introduce extreme memory and compute challenges. The KV cache for a single 128K-token conversation with Llama-3 70B reaches:

$$2 \times 80 \times 64 \times 128 \times 131072 \times 2 \approx \mathbf{344}\ \text{GB}$$

This dwarfs even the GPU's HBM capacity. Techniques for long-context efficiency:

- **Sparse attention:** Only attend to a subset of past tokens (e.g., StreamingLLM's "attention sinks")
- **KV cache quantization to INT2/INT4:** Sub-4-bit KV compression
- **KV cache offloading:** Spill to CPU memory or NVMe SSD
- **Chunked prefill:** Process long prompts in chunks to control peak memory

```python
from vllm import LLM, SamplingParams

llm = LLM(
    model="meta-llama/Llama-3-70b-instruct",
    max_model_len=131072,          # 128K context window
    enable_chunked_prefill=True,   # Process long prompts in chunks
    max_num_batched_tokens=2048,   # Process 2K tokens per prefill chunk
    tensor_parallel_size=4,        # Need 4 GPUs for 128K KV cache
)
```

Listing 11.1: Chunked prefill for long contexts in vLLM

11.2 Multimodal Inference

Models like LLaVA, Qwen-VL, and InternVL accept image inputs alongside text. The inference pipeline has an extra step: image encoding.

1. **Vision encoder:** A ViT (Vision Transformer) encodes the image into feature tokens — typically 256–1024 tokens per image.
2. **Projection:** A linear layer maps image features to the LLM's token space.
3. **LLM decode:** The LLM generates text given the image tokens and text prompt.

The bottleneck is that each image adds many tokens to the effective sequence length, inflating KV cache requirements significantly.

11.3 Agentic Inference

AI agents run multiple LLM calls in a loop: plan, act, observe, repeat. The infrastructure implications:

- **Long conversations:** Context accumulates over many steps. Efficient prefix caching is essential.
- **Tool calling:** The model generates structured JSON for tool calls. Low latency per step matters for responsiveness.
- **Parallel function calls:** Some frameworks call multiple tools simultaneously. The serving infrastructure must handle bursty traffic.
- **Request tracing:** Agents make many sub-requests. Tracing is essential for debugging and cost attribution.

Chapter Summary

Long-context inference strains KV cache memory and requires chunked prefill, sparse attention, or quantized KV. Multimodal models add a vision encoder step that contributes many extra tokens. Agentic workloads create bursty, highly correlated requests that benefit from aggressive prefix caching and low per-call latency.

APPENDIX A

CUDA Cheat Sheet

Essential CUDA API

Task	API	Notes
Allocate device memory	`cudaMalloc`, `cudaMallocAsync`	Use async variants with streams to reduce stalls.
Free device memory	`cudaFree`, `cudaFreeAsync`	Always free in reverse ownership order to simplify debugging.
Host/device transfer	`cudaMemcpy`, `cudaMemcpyAsync`	Minimize transfers; keep tensors on GPU between kernels.
Launch kernels	`kernel<<<grid, block, shmem, stream>>>`	Start with 256 threads/block for most inference kernels.
Thread indexing	`blockIdx`, `threadIdx`, `blockDim`	Global index in 1D: `idx = blockIdx.x * blockDim.x + threadIdx.x`.
Synchronization	`cudaDeviceSynchronize`, `cudaStreamSynchronize`, `__syncthreads()`	Prefer stream sync over device-wide sync in production.
Error handling	`cudaGetLastError`, `cudaGetErrorString`	Wrap all CUDA calls in a check macro.
Device query	`cudaGetDeviceProperties`	Read SM count, shared memory, max threads/block before tuning.

Minimal Kernel Checklist

- Validate bounds: every kernel should guard out-of-range threads.
- Use contiguous memory access patterns for coalescing.
- Reuse data in shared memory when read multiple times.
- Avoid warp divergence in hot paths.
- Profile before optimizing; trust metrics, not intuition.

Performance Rules of Thumb

- Use at least 128 threads per block; 256 is usually optimal
- Ensure memory accesses are coalesced (stride-1 access by consecutive threads)
- Avoid warp divergence in performance-critical paths
- Use shared memory for data reused >2 times within a block
- Overlap compute and memory transfers using CUDA streams
- Profile with nsys before optimizing — never guess the bottleneck

APPENDIX B

Linux Commands for AI Engineers

Essential Linux Command Map

Goal	Recommended Command	Why It Matters
Check GPU health	`nvidia-smi`	Fast snapshot of util, memory, thermals.
Watch GPU over time	`nvidia-smi dmon -s u` or `nvtop`	Detect bursts and stalls that averages hide.
Inspect model disk size	`du -sh /models/*`	Prevent slowdowns from disk pressure.
Check free memory	`free -h`	Spot host-memory pressure before OOM.
Verify API port	`ss -tlnp | grep 8000`	Confirms serving process is bound and reachable.
Validate health endpoint	`curl http://localhost:8000/health`	Quick liveness check for probes and load balancers.
List loaded models	`curl http://localhost:8000/v1/models`	Verifies runtime model registration.
Benchmark quickly	`time python inference_benchmark.py`	Baseline latency and throughput before tuning.
Control visible GPUs	`CUDA_VISIBLE_DEVICES=0,1`	Isolate workloads and avoid noisy neighbors.
Keep jobs alive	`tmux new -s inference`	Run long jobs safely over unstable sessions.

APPENDIX C

Benchmarking Toolkit

Benchmark Design Template

Category	Field	Recommended Default
Endpoint	URL	`http://localhost:8000/v1/chat/completions`
Traffic shape	Concurrent users	20 (start), then sweep 1, 5, 10, 20, 50
Workload	Input length	512 prompt tokens (plus a long-prompt run at 4K)
Workload	Output length	128 and 512 token profiles
Sampling	Temperature	0.0 for deterministic latency comparisons
Duration	Requests per run	Minimum 200 requests per configuration
Warmup	Warmup requests	20 warmup calls before timing
Observability	Metrics captured	TTFT p50/p95/p99, TPOT p50/p95, tok/s, error rate
Stability	Retry policy	Retry once on timeout, log all failures

Core Metrics

$$\text{TTFT} = t_{\text{first token}} - t_{\text{request start}}$$

$$\text{TPOT} = \frac{t_{\text{last token}} - t_{\text{first token}}}{N_{\text{output}} - 1}$$

$$\text{Throughput (tok/s)} = \frac{\sum N_{\text{output}}}{\text{total wall time (s)}}$$

Runbook

1. Warm up the runtime and model weights.
2. Execute fixed-shape tests (same prompt and output length).
3. Sweep concurrency and collect percentile metrics.
4. Repeat each configuration at least 3 times.
5. Report median of runs and attach p95/p99 tails.

APPENDIX D

GPU Profiling Tools

Profiling Workflow

Step	Focus	Primary Tool / Command	What to Look For
1	Health snapshot	`nvidia-smi`	GPU utilization, memory usage, temperature, and power draw.
2	Timeline bottlenecks	`nsys profile -trace=cuda,nvtx,osrt,cudnn,cublas -sample=cpu -output=my_profile python my_inference.py`	Idle gaps between kernels, large memcpy phases, weak CPU/GPU overlap.
3	Kernel deep dive	`ncu -set full -kernel-name cutlass -launch-skip 10 -launch-count 3 -o kernel_analysis python run_matmul.py`	SM active cycles, memory throughput, L2 hit rate, warp stalls.
4	Efficiency score	MFU estimation	Compare achieved FLOP/s against hardware peak to separate compute-limit vs IO-limit behavior.
5	KV cache behavior	`curl http://localhost:8000/metrics | grep cache`	`vllm:gpu_cache_usage_perc` below 90%, hit rate trending upward.

Profiling Interpretation Playbook

- If GPU utilization is low and queue depth is low: increase effective batch size.
- If TTFT spikes but TPOT is stable: investigate prefill pressure and scheduler fairness.
- If TPOT rises with long prompts: decode is likely memory-bound; test quantization or shorter max output.
- If memory is consistently above 95%: reduce sequence length, lower max concurrent sequences, or add GPUs.
- If throughput is flat while latency worsens: you are saturation-limited; scale horizontally.

MFU Reference

$$\text{MFU} = \frac{\text{achieved FLOP/s}}{\text{peak FLOP/s}}$$

Practical rule: for inference, MFU above 30% is generally healthy for memory-heavy decode; compute-heavy prefill can exceed 50% when tuned.

Interpreting Common Profiling Issues

Symptom	Likely Cause	Fix
GPU util $< 30\%$	Batch too small	Increase max_num_seqs
Memory $> 95\%$	KV cache overflow	Reduce seq length or add GPUs
TTFT spikes	Large prefill batches	Enable chunked prefill
TPOT high	Memory bandwidth bound	Normal for small batches; use quantization
Frequent OOM	Memory fragmentation	Use PagedAttention (vLLM)
Slow decode after long prefill	KV cache eviction	Increase GPU memory fraction

Table D.2: Common Profiling Issues and Fixes

APPENDIX E

Research Papers Every Inference Engineer Should Read

Foundational Papers

Attention Is All You Need (Vaswani et al., 2017) The original Transformer paper. Understand this before anything else. https://arxiv.org/abs/1706.03762

FlashAttention (Dao et al., 2022) IO-aware exact attention that became standard in all production systems. https://arxiv.org/abs/2205.14135

FlashAttention-2 (Dao, 2023) Further optimizations: 2× speedup, better parallelism. https://arxiv.org/abs/2307.08691

Efficient Memory Management for LLM Serving with PagedAttention (Kwon et al., SOSP 2023) Foundation of vLLM. Must-read for KV cache management. https://arxiv.org/abs/2309.06180

Quantization

LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale (Dettmers et al., 2022) First practical INT8 quantization for LLMs. https://arxiv.org/abs/2208.07339

GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers (Frantar et al., 2022) State-of-the-art INT4 PTQ. https://arxiv.org/abs/2210.17323

AWQ: Activation-aware Weight Quantization (Lin et al., 2023) Better accuracy than GPTQ at same bit width. https://arxiv.org/abs/2306.00978

SmoothQuant (Xiao et al., 2022) Smooth activation outliers for INT8 quantization. https://arxiv.org/abs/2211.10438

Speculative Decoding

Accelerating Large Language Model Decoding with Speculative Sampling (Chen et al., 2023) Original speculative decoding paper. https://arxiv.org/abs/2302.01318

EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty (Li et al., 2024) 3× speedup with draft head sharing target model features. https://arxiv.org/abs/2401.15077

Parallelism and Distributed Inference

Megatron-LM (Shoeybi et al., 2019) Efficient tensor and pipeline parallelism for large models. https://arxiv.org/abs/1909.08053

Alpa: Automating Inter- and Intra-Operator Parallelism (Zheng et al., 2022) Automatic parallelism for large models. https://arxiv.org/abs/2201.12023

Systems and Scheduling

Orca: A Distributed Serving System for Transformer-Based Generative Models (Yu et al., OSDI 2022) First continuous batching paper for LLM serving. https://www.usenix.org/conference/osdi22/presentation/yu

SGLang: Efficient Execution of Structured Language Model Programs (Zheng et al., 2024) RadixAttention for prefix sharing. https://arxiv.org/abs/2312.07104

DejaVu: Contextual Sparsity for Efficient LLMs at Inference Time (Liu et al., 2023) Dynamic sparsity for faster inference. https://arxiv.org/abs/2310.17157

2025 Frontier Papers

TokenWeave: Reducing Tensor-Parallel Communication Overhead (arXiv 2505.11329, 2025) 1.2× latency improvement in tensor-parallel low-latency inference.

QuantSpec: Self-Speculative Decoding with Hierarchical Quantized KV Cache (arXiv 2502.10424, 2025) 1.78× throughput via combined speculative decoding and KV quantization.

GPU-Accelerated INT8 Quantization for KV Cache Compression (arXiv 2601.04719, 2026) 4× memory reduction with CUDA kernels achieving 1694× CPU speedup.

Index